Unit I Introduction to Algorithm and Data Structures (07 Hours)

Unit I - Syllabus

Unit I – Introduction to Algorithms and Data Structures (7 Hours)

From Problem to Program (Problem, Solution, Algorithm, Data Structure and Program). Data, Information, Knowledge and Data Structure; Abstract Data Types (ADT); classification (Linear/Non-linear, Static/Dynamic, Persistent/Ephemeral). Algorithms: problem solving, characteristics, design tools (pseudo-code, flowchart). Complexity: space and time complexity, asymptotic notation (Big-O, Theta, Omega), step-count method, analysis of Linear, Quadratic, Cubic and Logarithmic constructs.

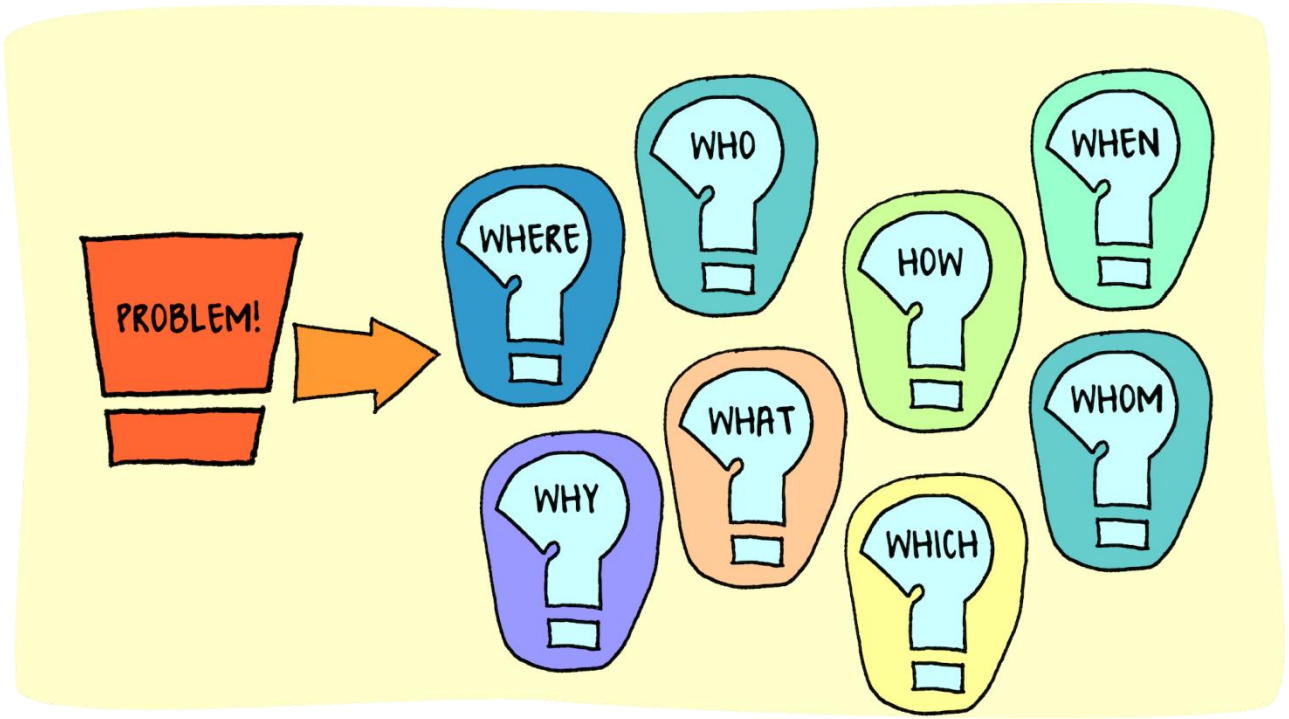
Algorithmic strategies:- Divide and Conquer, Greedy.



Introduction

From Problem to Data Structure (Problem, Logic, Algorithm, and Data Structure).

What is Problem Solving and Why is it Important?



What is Problem Solving and Why is it Important?

- The ability to solve problems is a basic life skill and essential to our day-to-day lives, at home, at school, and at work.
- We solve problems every day without really thinking abouthow we solve them.
- For example: it's raining and you need to go to the store. What do you do? There are lots of possible solutions. Take your umbrella and walk. If you don't want to get wet, you can drive, or take the bus. You might decide to call a friend for a ride, or you might decide to go to the store another day. There is no right way to solve this problem and different people will solve it differently.

Problem Solving

- "Problem is defined as situation or condition which needs to solve to achieve goal
- ***Problem solving is the process of identifying a problem, developing possible solution paths, and taking the appropriate course of action.***

- Problem solving is the application of ideas, skills, or factual information to achieve the solution to a problem or to reach a desired outcome.

Why should we learn Data Structures?

- Data Structures and Algorithms are two of the key aspects of Computer Science.
- Data Structures allow us to organize and store data, whereas Algorithms allow us to process that data efficiently and meaningfully.
- Learning Data Structures and Algorithms will help you become better Programmers.
- You will be able to write code that is effective and reliable.
- You will also be able to solve problems more quickly and efficiently.

Data Structure Definition

- A data structure is **a way of organizing and storing data in a computer** so that it can be accessed and used efficiently.

- A **data structure** is a specialized format for organizing, processing, storing, and managing data in a computer so that it can be accessed and modified efficiently

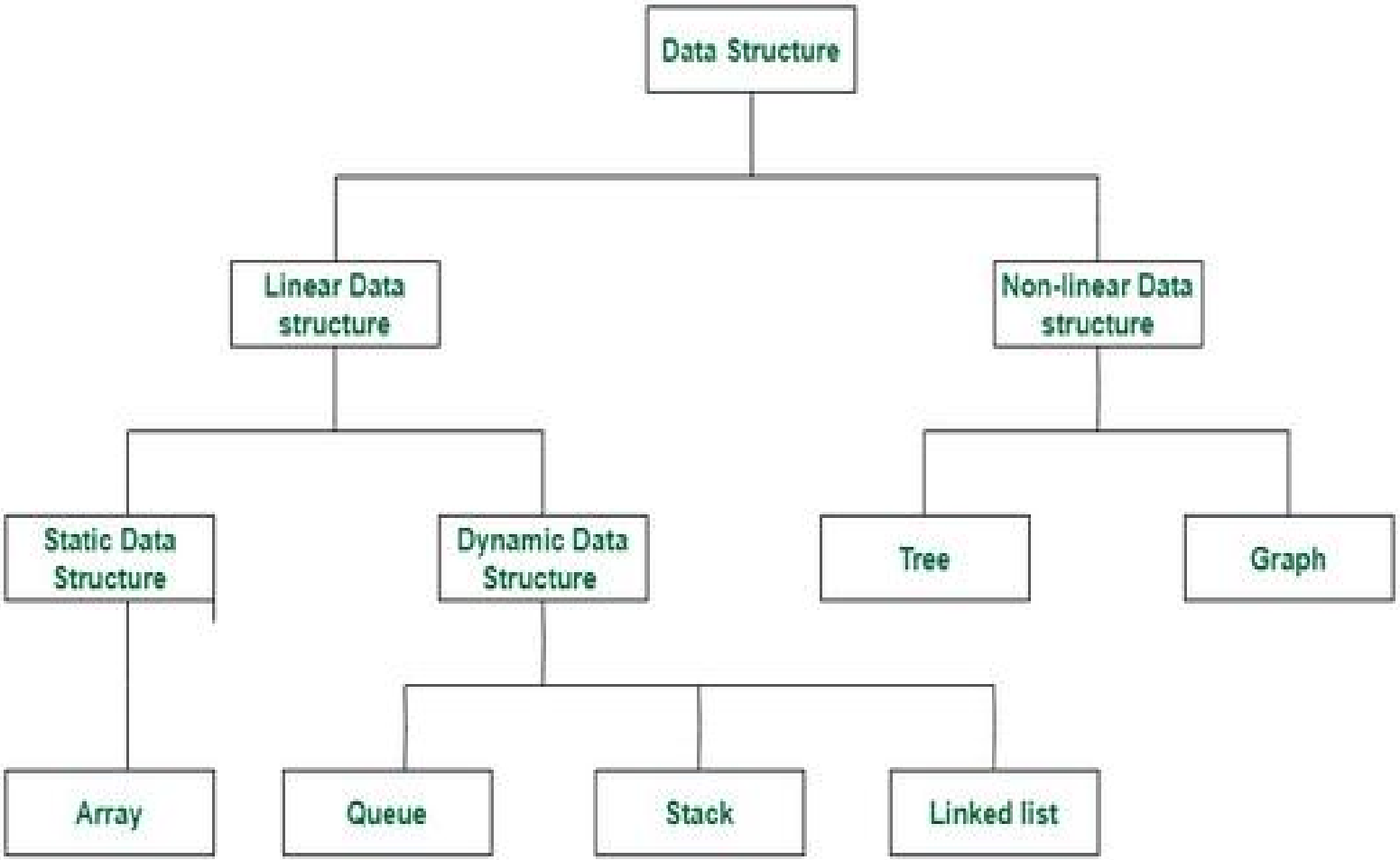
- It refers to the logical or mathematical representation of data, as well as the implementation in a computer program.

Data structures can be classified into two broad categories:

Linear Data Structure: A data structure in which data elements are arranged sequentially or linearly, where each element is attached to its previous and next adjacent elements, is called a linear data structure. Examples are array, linked list, stack, queue, etc.

Non-linear Data Structure: Data structures where data elements are not placed sequentially or linearly are called non-linear data structures. Examples are trees and graphs.

Classification of Data Structure



Basic Terms

- **Data** – Data are values or set of values.
- **Data Item** – Data item refers to single unit of values.
- **Group Items** – Data items that are divided into sub items are called as Group Items.
- **Elementary Items** – Data items that cannot be divided are called as Elementary Items.
- **Attribute and Entity** –An entity is that which contains certain attributes or properties, which may be assigned values.
- **Entity Set** – Entities of similar attributes form an entity set.
- **Field** – Field is a single elementary unit of information representing an attribute of an entity.
- **Record** – Record is a collection of field values of a given entity.
- **File** – File is a collection of records of the entities in a given entity set.

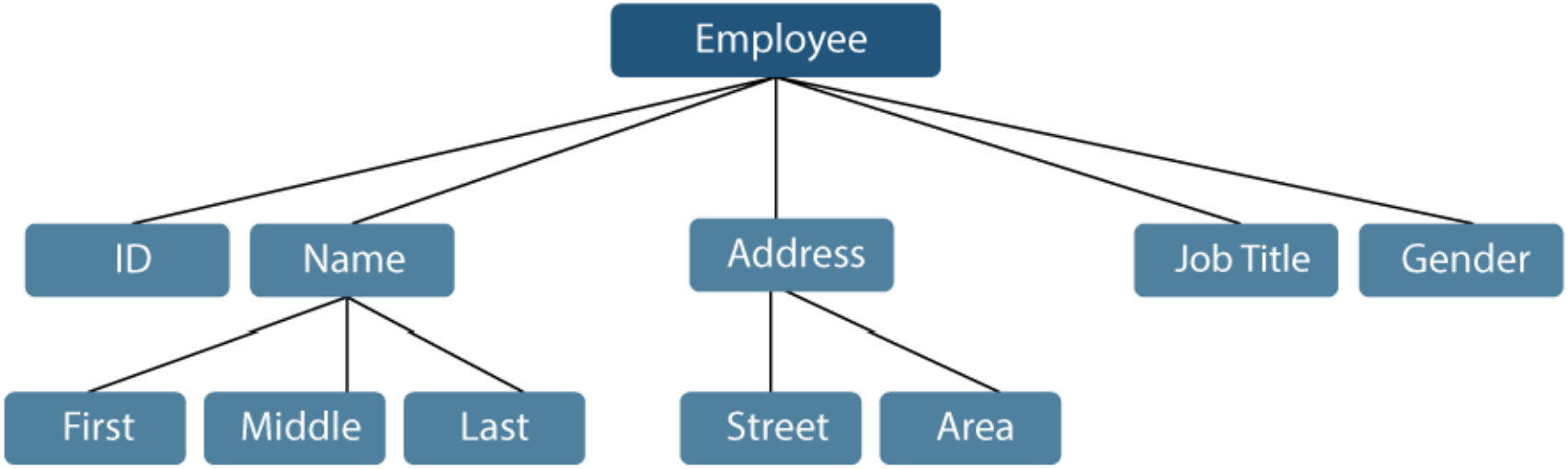
Data

Data, Information, Knowledge, and Data structure, Abstract Data Types (ADT),

Data

- **Data**" to refer to Computer Information, either transmitted or stored
- Data isa raw and unorganized fact that is required to be processed to make it meaningful.
- Data are individual units of information
- In analytical processes, data are represented by variables.
- Data is always interpreted, by a human or machine, to derive meaning. So, data is meaningless. Data contains numbers, statements, and characters in a raw form.

Example



In the example mentioned above, the items such as ID, Age, Gender, First, Middle, Last, Street, Locality, etc., are elementary data items. In contrast, the Name and the Address are group data items.

Information

- Information is processed, organized, and structured data.
- We use the information in significant ways.
- A researcher can evaluate their hypothesis on the basis of collected data. A businessman analyzes, organizes the collected data, and makes business decisions.

Data and information

Data	Information
Data refers to raw facts that have no meaning	Information refers to processed data that has purpose and meanings.
Data is independent of information	Information is dependent on data
Data is not enough to take a decision.	The information is sufficient to help to make a decision in the respective context
In raw form and unprocessed and unstructured	Processed and structured
We cannot make a decision based on the data	Meaningful data, i e Information can be used to make decisions.
Data is normally huge in volume	Information is normally short in volume
Data is not always useful as it is needed to be processed	Information is always useful as it provides some meaning

Knowledge

- **Knowledge** is what we know.
- Knowledge is unique to each individual and is the accumulation of past experience and insight that shapes the lens by which we interpret, and assign meaning to, information.
- For knowledge to result in action, an individual must have the authority and capacity to make and implement a decision.
- Knowledge (and authority) are needed to produce **actionable information** that can lead to impact.

Data, Information and Knowledge

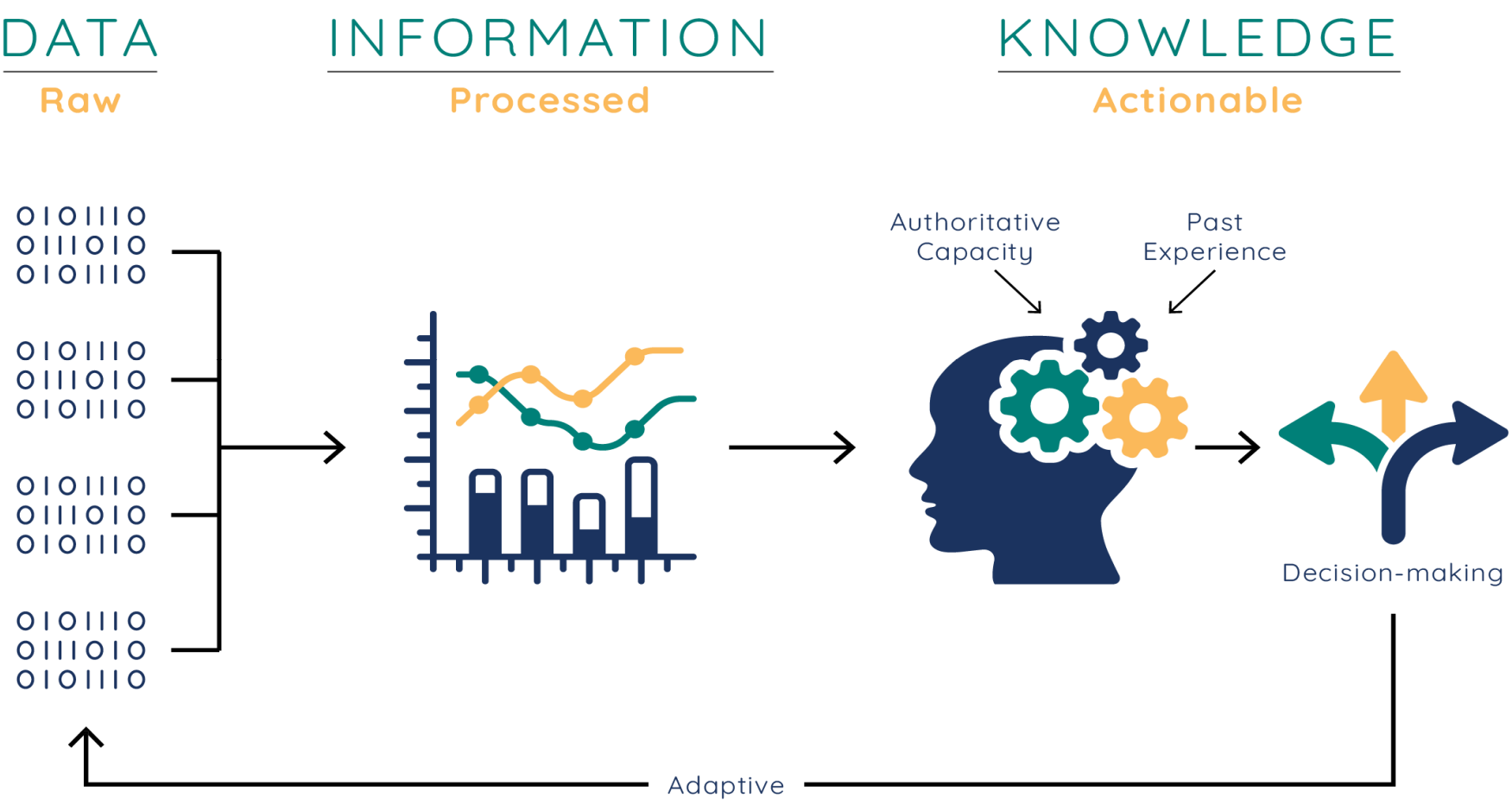
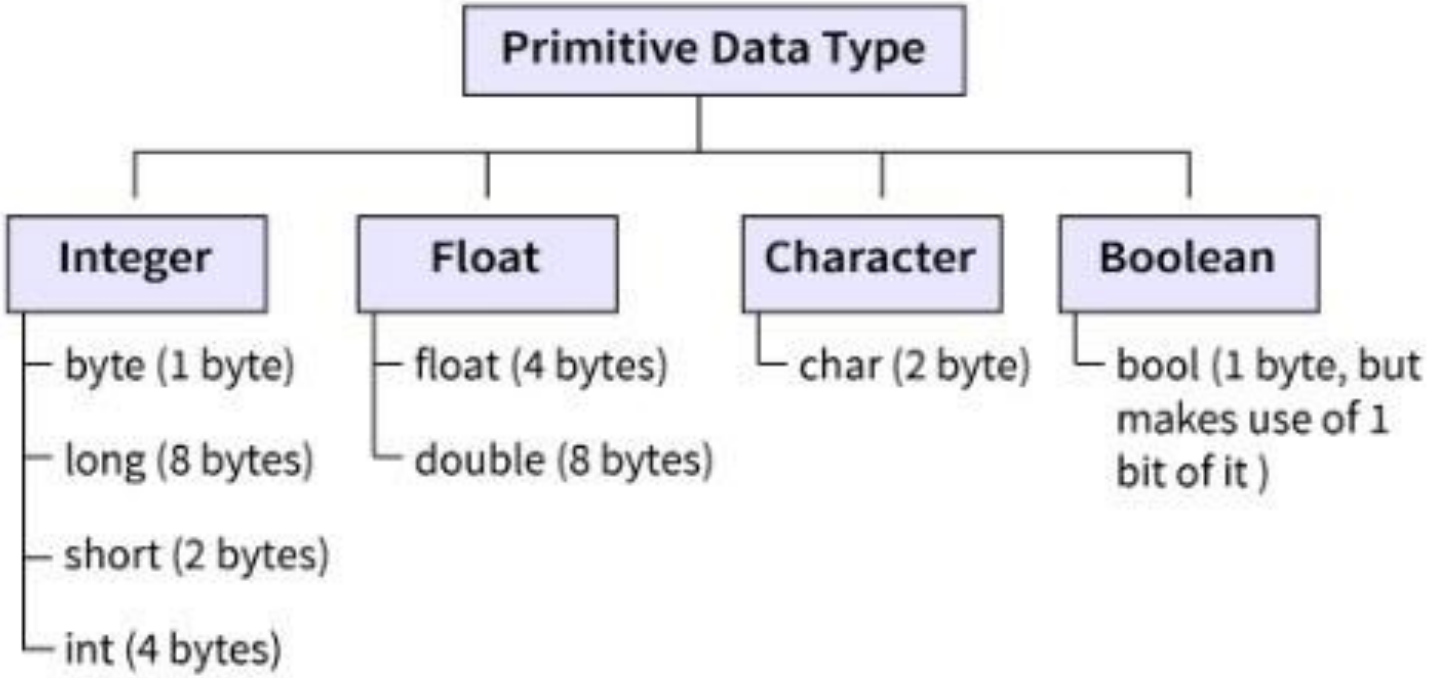


Figure 1:The flow from data to information to knowledge.

Data Types

- Data type refers to the kind of data a variable may store. Whenever we try to implement any algorithm in some programming language, we need variables.
- A variable may have any value as per the facilities provided by that language. Data type is a term that specifies the type of data that a variable may hold in the programming language.
  - Built in Data Type-int ,float ,char etc.
  - User Defined Data Type : Class, structure, and union

Data Type and their Size



Data Structure

- Data Structure is a particular way of storing and organizing data in the memory of the computer so that these data can easily be retrieved and efficiently utilized in the future when required.
- The data can be managed in various ways, like the logical or mathematical model for a specific organization of data is known as a data structure.

Abstract Data Types (ADT)

- To simplify the process of solving problems, we can create data structures along with their operations, and such data structures that are not in-built are known as Abstract Data Type (ADT).
- Abstract Data type (ADT) is a type (or class) for objects whose behavior is defined by a set of values and a set of operations. The definition of ADT only mentions what operations are to be performed but not how these operations will be implemented. It does not specify how data will be organized in memory and what algorithms will be used for implementing the operations. It is called "abstract" because it gives an implementation-independent view.

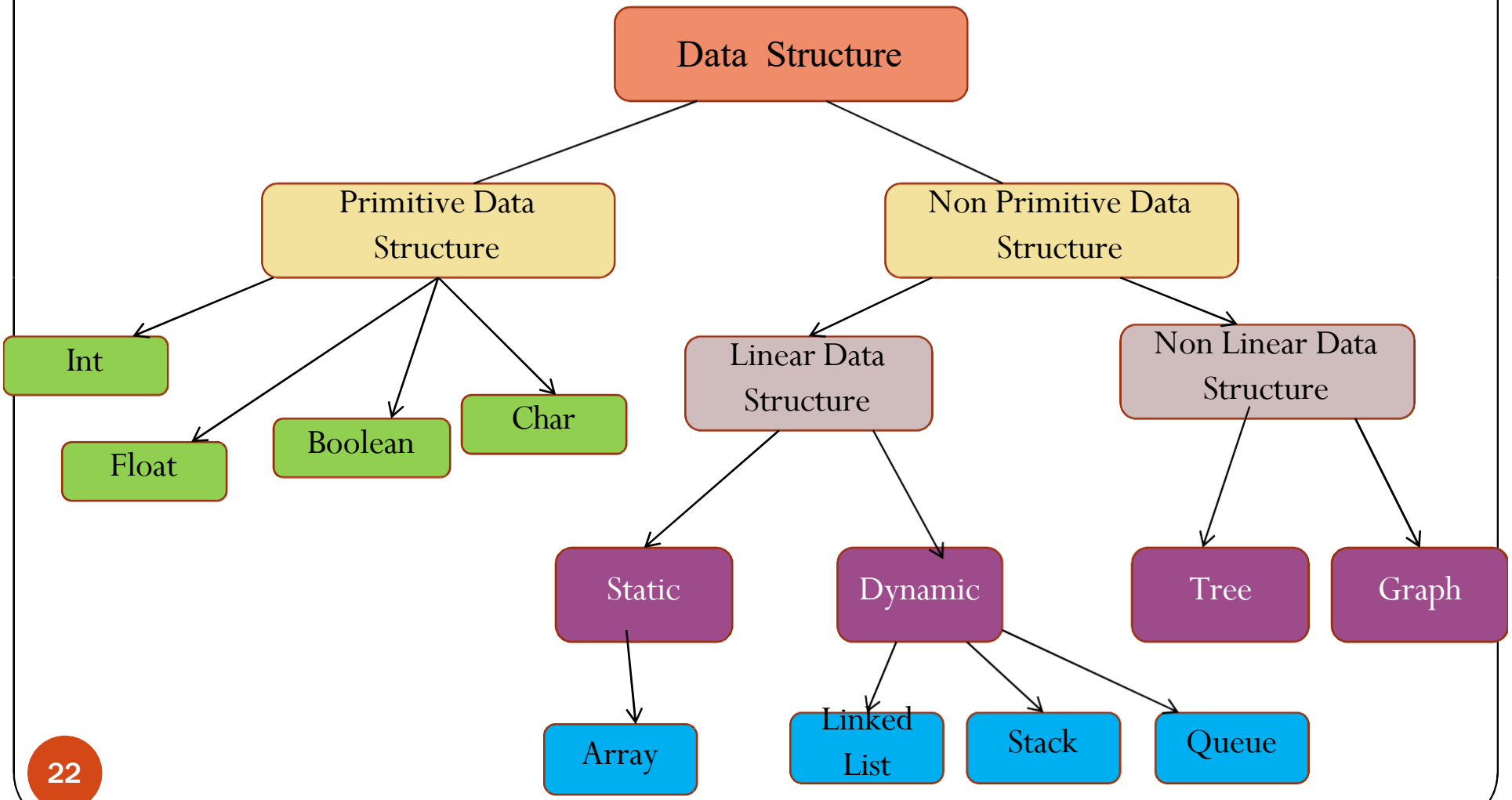
Algorithm

- An algorithm is a collection of steps to solve a particular problem. Learning data structures and algorithms allow us to write efficient and optimized computer programs.

Data Structure Classification/ Types of Data Structure

Linear and Non-linear, Static and Dynamic, Persistent and Ephemeral data structures

Data Structure Classification/ Types of Data Structure



Primitive and Non-primitive Data Structures

- Primitive data structures are the fundamental data types which are supported by a programming language. Eg. integer, real, character, and boolean.
- Non-primitive data structures are those data structures which are created using primitive data structures. Eg. linked lists, stacks, trees, and graphs, class, structure
- Non-primitive data structures can further be classified into two categories: linear and non-linear data structures.

Primitive VS Non Primitive Data Structure

Primitive Data Structure	Non-Primitive Data Structure
Primitive data structures store single data type values.	Non-primitive data structures store multiple data type values.
Examples- Integer, Boolean, Character, Float, etc.	**Examples-** Array, Linked List, Stack, etc.
Primitive data structures cannot store NULL values.	Non-primitive data structures can store NULL values.
The size of primitive data structures depends on their type.	The size of non-primitive data structures is not fixed.

Linear and Non-linear Structures

- If the elements of a data structure are stored in a linear or sequential order, then it is a linear data structure. Eg. arrays, linked lists, stacks, and queues
- If the elements of a data structure are not stored in a sequential order, then it is a non-linear data structure. Eg. Tree and Graphs.
- In non-linear data structures, every data element may have more than one predecessor as well as successor. Elements do not form any particular linear sequence.
- Non-linear data structures are generally implemented in the same way as linked lists. Hence, trees and graphs can be implemented as dynamic data structures.

Linear VS Non Linear

Linear Data Structures	Non Linear Data Structures
Linear Data Structures used to represent sequential data.	Non Linear Data Structures used to represent hierarchical data.
Linear Data Structures are easy to implement.	Non Linear Data Structures are difficult to implement
Implemented using array and linked list	Mostly implemented using linked list
These are mostly used in application of software development	These are mostly used in application of graph theory, AI, Image processing
For implementation of linear DS we don't need Non Linear Data Structure	For implementation of Nonlinear DS we need Linear Data Structure
Eg. List, Stack,Queue	Eg.Trees and Graph

Static and Dynamic Data Structures

- A data structure is referred to as a static data structure if it is created before program execution begins (also called during compilation time). The variables of static data structure have user-specified names. **Eg. Array**
- A data structure that is created at run-time is called dynamic data structure. The variables of this type are not always referenced by a user-defined name. These are accessed indirectly using their addresses through pointers. **Eg. Linked List, Stack, Queue**

Static VS Dynamic Data Structures

Static Data Structures	Dynamic Data Structures
A data structure whose size is fixed when it is created in memory	Data structure which will extend and change its size to fit the data
Inefficient as memory is allocated that is not needed	Efficient as the amount of memory varies
Fast access to the data as memory location is fixed when the program is written	Slower access to each element as the memory location is allocated at run time
Eg. Array	Eg. Linked List

Persistent and Ephemeral Data Structures

- A data structure that supports operations on the most recent version as well as the previous version is termed as a persistent data structure. Eg. Magnetic media, such as hard disk drives and tape.
- A data structure is **partially persistent** if all versions can be accessed but only the newest version can be modified. **Fully persistent** if every version can be both accessed and modified. **Confluently persistent** is when we merge two or more versions to get a new version.
- An ephemeral data structure is one that supports operations only on the most recent version. Eg.Linked List, array

Ephemeral data structures are the **standard, traditional** data structures we use in most programs.

Updates are destructive — once you change them, the old version is lost.

- **Key Characteristics:**

In-place modification

No access to previous versions after update

Used in **imperative programming languages** (like C, Java)

Example in C:

```
int arr[] = {1, 2, 3};
arr[0] = 10; // old value 1 is overwritten
```

After the update, there's **no way to access** the original array {1, 2, 3}.

2. Persistent Data Structures

- **Definition:**

A **persistent data structure** keeps **previous versions** accessible even after updates.

Instead of modifying the structure directly, it returns a **new version** while keeping the old one unchanged.

- **Types:**

Partial Persistence: Can access old versions but only modify the latest.

Full Persistence: Can access and modify any version.

Confluent Persistence: Can merge multiple versions.

- **Key Characteristics:**

•Non-destructive updates

•Common in **functional programming** (Haskell, Clojure)

•Efficient version tracking

- **Example (in pseudocode):**

pseudo

v1 = [1, 2, 3]
v2 = v1.update(0, 10) // v2 = [10, 2, 3], v1 = [1, 2, 3]
Here, both v1 and v2 **coexist**.

Feature	Ephemeral	Persistent
Updates	Destructive (overwrites)	Non-destructive (creates new)
Access to previous versions	No	Yes
Memory usage	Less	More (but can be optimized)
Typical usage	C, Java, Python	Haskell, Clojure, Git, Blockchain
Performance trade-off	Faster for single version	Slower but version-safe

Algorithm

Problem Solving, Introduction to algorithm, Characteristics of algorithm, Algorithm design tools: Pseudo-code and flowchart

Problem Solving

- A problem-solving algorithm is a procedure that is guaranteed to produce a solution if it is followed strictly.

- Problem solving is the act of defining a problem; determining the cause of the problem identifying prioritizing and selecting alternatives for a solution; and implementing a solution.The problem-solving process.



Classification of Problems

1) **Problems based on Algorithmic Solutions:**
   Include Step by step Instructions and Series of actions
   eg: To find Fibonacci series of steps/actions are required.

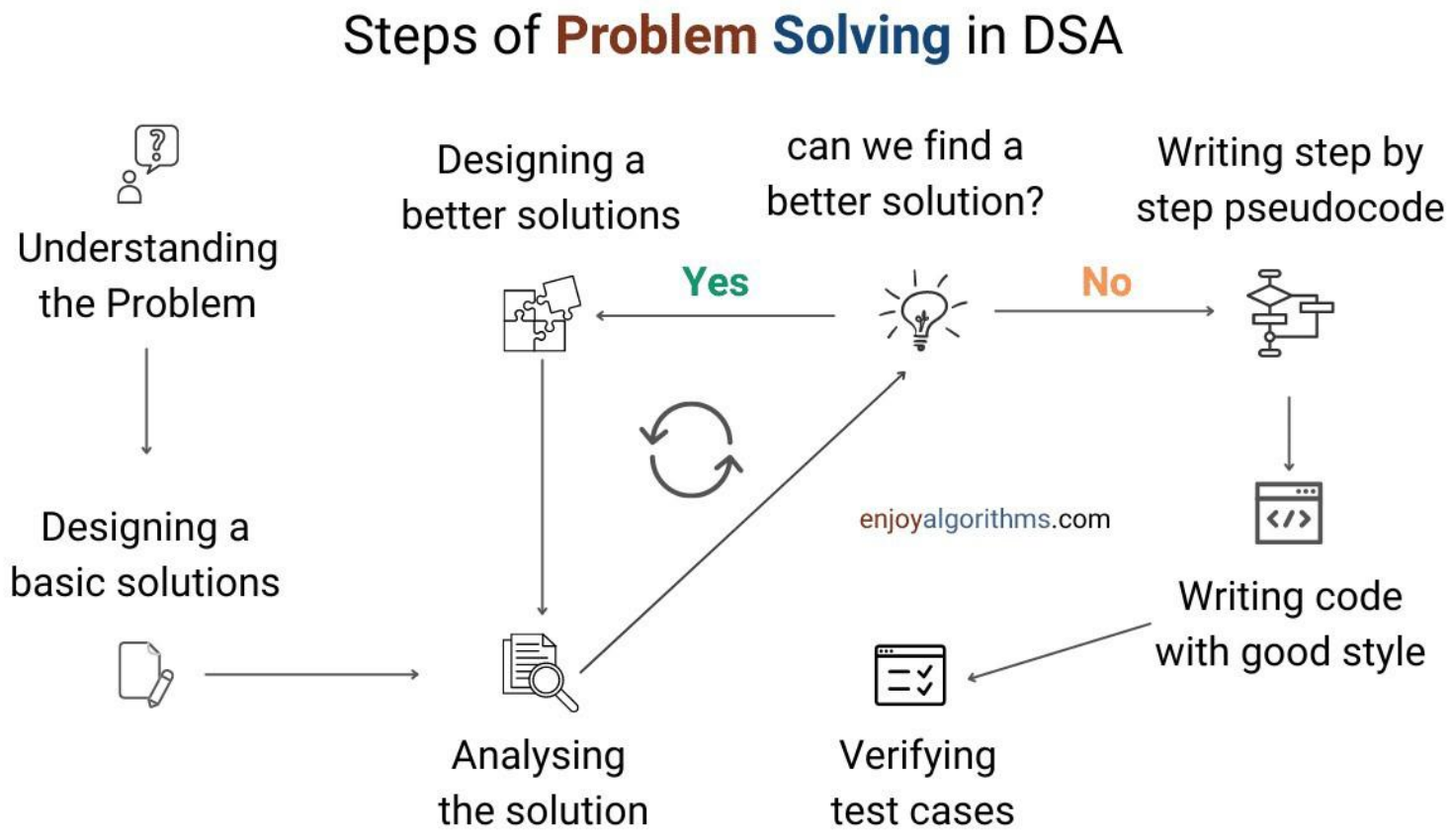
2) **Problems based on Heuristic Solutions:**
   Trial and Error form, No sequence of actions or instructions

eg:

1)**Common Sense Heuristic**: If it is raining outside, you should bring an umbrella

2) **Familiarity Heuristic :**Consumers buy the same brands over and over regardless of the quality of the products

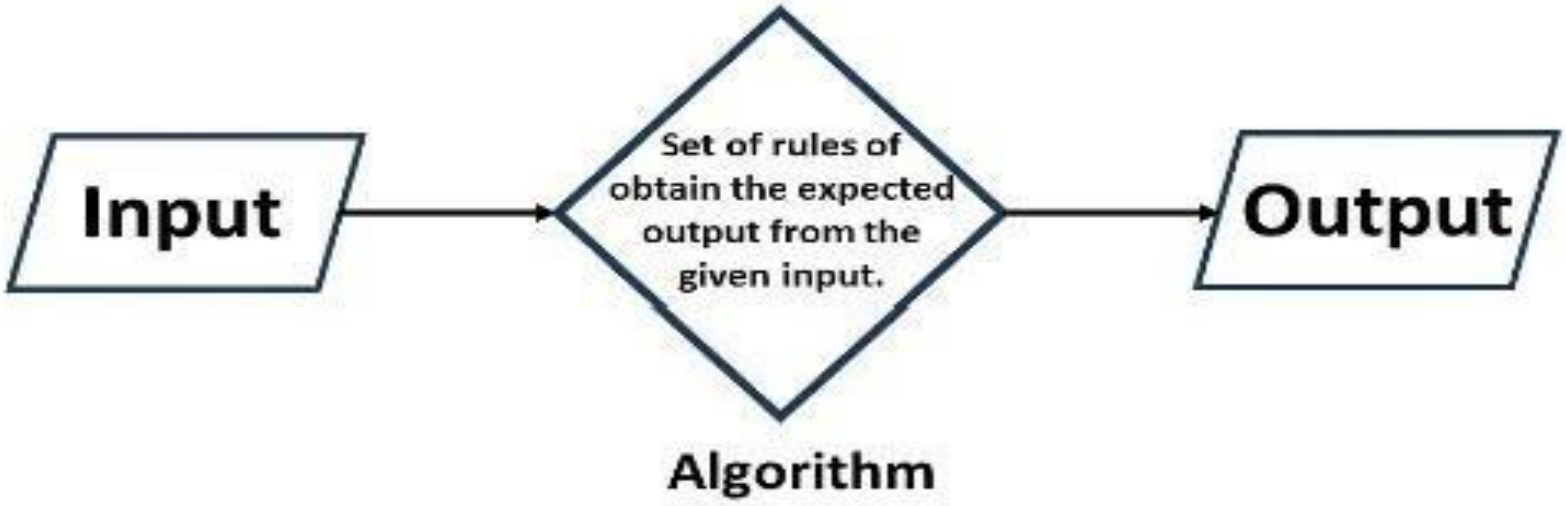
Steps of Problem solving in DSA



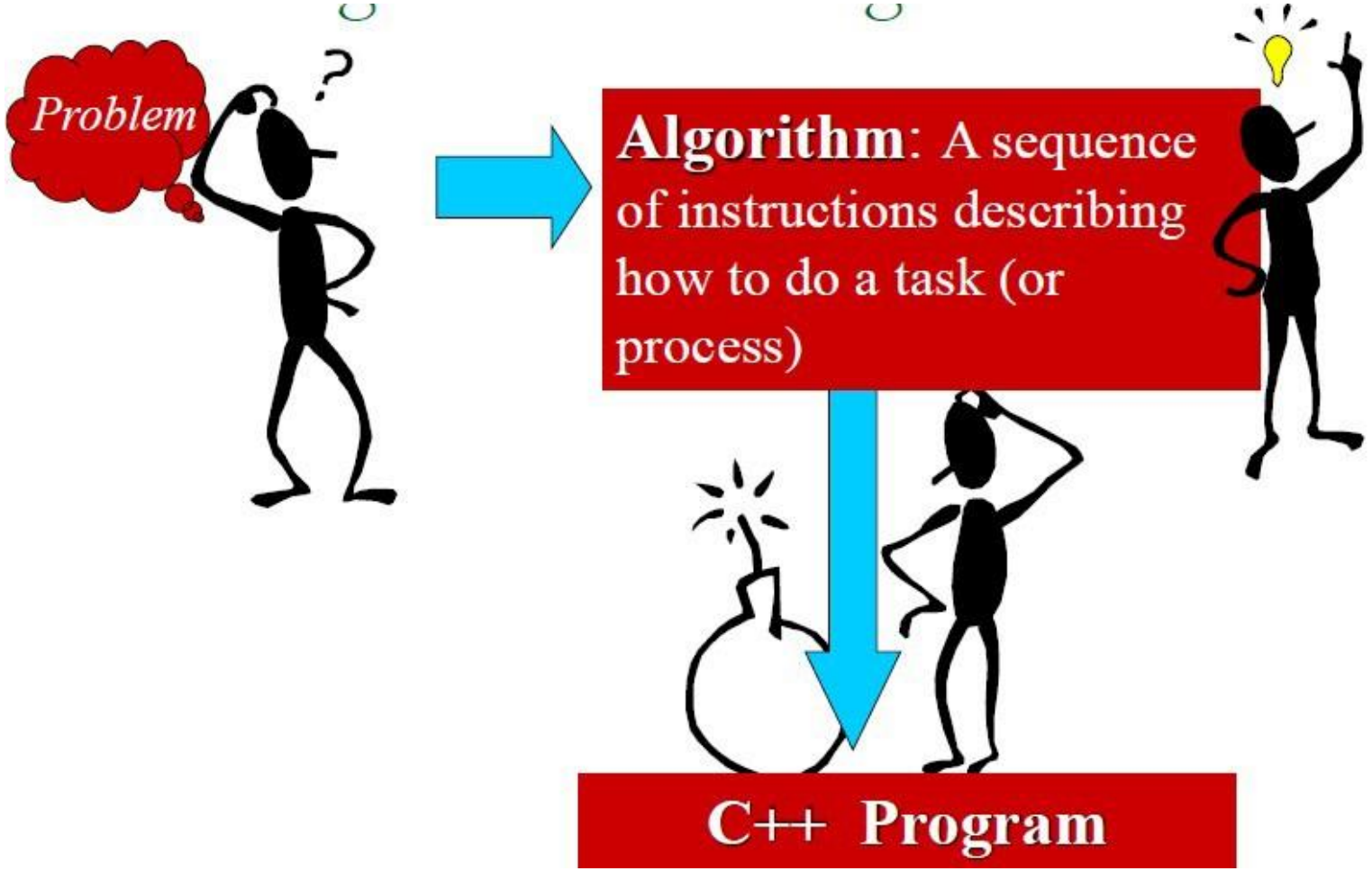
Introduction to Algorithm

- An algorithm is simply a **set of steps used to complete a specific task**.
- An algorithm is a set of step-by-step procedures, or a set of rules to follow, for completing a specific task or solving a particular problem. algorithm is **a set of rules that specify the order and kind of arithmetic operations that are used on a specified set of data**.
- An algorithm is an effective method for solving a problem using a finite sequence of instructions.

Algorithm


Input
Set of rules of obtain the expected output from the given input.
Output
Algorithm

From Algorithm to process



Characteristics of Algorithm

- **Input** – An algorithm should have 0 or more well-defined inputs.
- **Output** – An algorithm should have 1 or more well-defined outputs, and should match the desired output.
- **Finiteness** – Algorithms must terminate after a finite number of steps.
- **Feasibility** – Should be feasible with the available resources.
- **Independent** – An algorithm should have step-by-step directions, which should be independent of any programming code.
- **Unambiguous** – Algorithm should be clear and unambiguous. Each of its steps (or phases), and their inputs/outputs should be clear and must lead to only one meaning.

Algorithm and Program

Algorithm:

```
Average (list, size_of_list):
{
        Sum => 0;

        For each item x in list
        begin
                Sum => Sum+x;
        end

        Avg => Sum / size_of_list ;

        return Avg;
}
```

Program:

```
float Average(int arr [], int sizeOfArr) {
   float sum = 0;

   for(int i = 0; i < sizeOfArr; i++) {

      sum = sum + arr[i];

   }

   float avg = sum/sizeOfArr;

   return avg;
}
```

Pseudo code

- **Pseudocode** is a method of describing computer algorithms using a combination of natural language and programming language. It is essentially an intermittent step towards the development of the actual code.

Pseudo code for addition of two numbers

- **Pseudocode to Add Two Numbers**

BEGIN.

NUMBER s1, s2, **sum**.

OUTPUT("Input number1:")

INPUT s1.

OUTPUT("Input number2:")

INPUT s2.

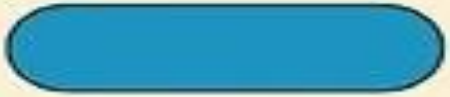
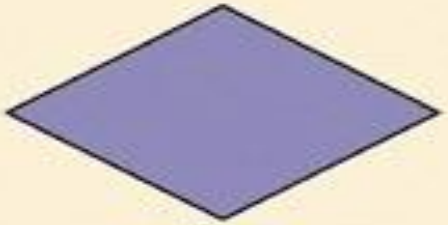
sum=s1+s2.

OUTPUT **sum**.

Flowchart

- A flowchart is a schematic representation of an algorithm or a stepwise process, showing the steps as boxes of various kinds, and their order by connecting these with arrows. Flowcharts are used in designing or documenting a process or program.

- A flow chart, or flow diagram, is a graphical representation of a process or system that details the sequencing of steps required to create output.

Name	Symbol	Use in flowchart
Oval		Denotes the beginning or end of a program.
Flow line		Denotes the direction of logic flow in a program.
Parallelogram		Denotes either an input operation (e.g., INPUT) or an output operation (e.g, PRINT).
Rectangle		Denotes a process to be carried out (e.g., an addition).
Diamond		Denotes a decision (or branch) to be made. The program should continue along one of two routes (e.g., IF/THEN/ELSE).
Circle		Connector . Connects two segment of a flow-chart when segments are on different pages

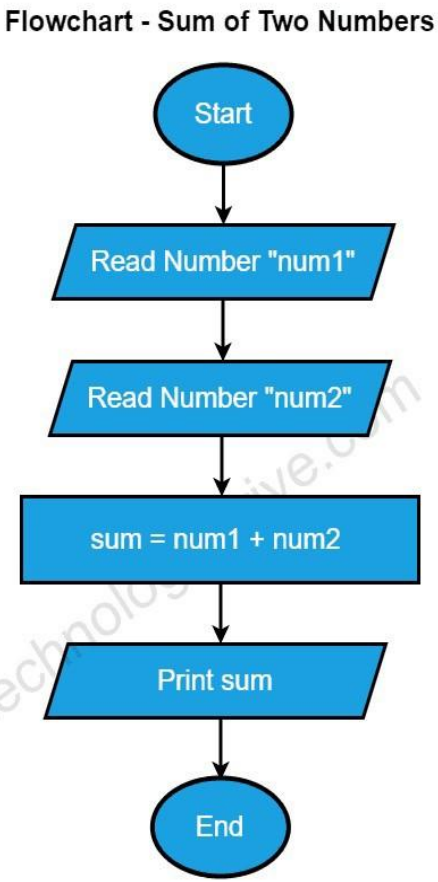
Symbol	Name	Function
	Start/end	An oval represents a start or end point
	Arrows	A line is a connector that shows relationships between the representative shapes
	Input/Output	A parallelogram represents input or output
	Process	A rectagle represents a process
	Decision	A diamond indicates a decision

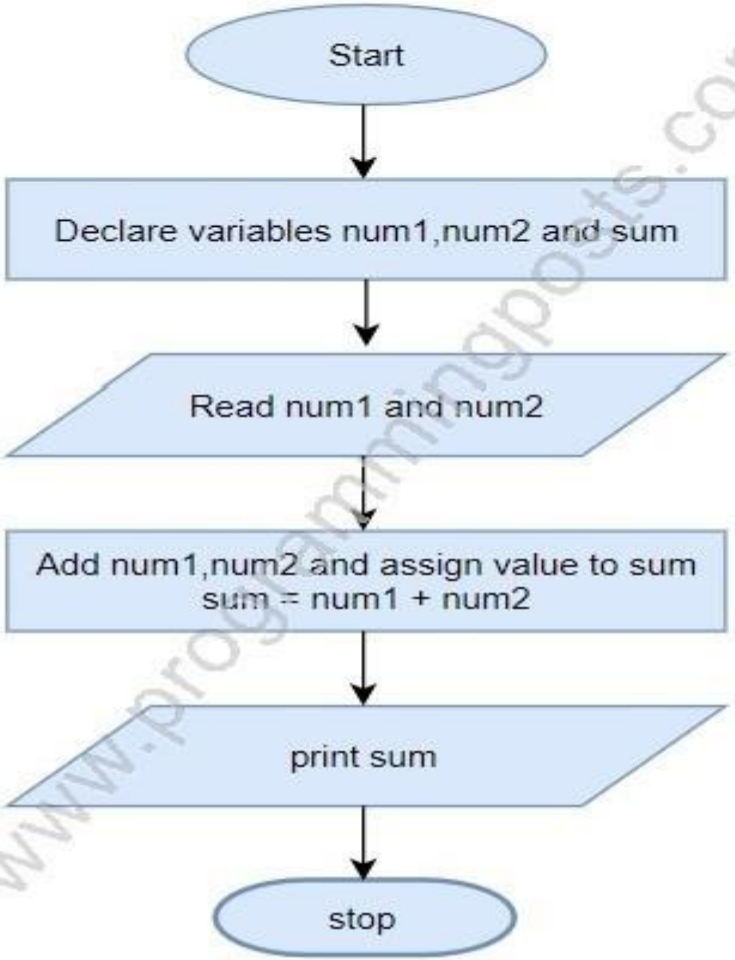
Algorithm for addition of two number

- Input : Two numbers num1 and num2
- Step 1: Start
- Step 2: Declare sum to 0 (This is optional step, during step5 we can add declaration and assign directly as well)
- Step 3: Read number num1
- Step 4: Read number num2
- Step 5: Add num1 and num2 and assign result to variable sum
- Step 6: Print sum
- Step 7: Stop
- Output: Sum of num1 and num2

Flowchart for addition of two number

technologystrive.com





technologystrive.com
technologystrive.com
technologystrive.com
© www.programmingposts.com

Complexity of algorithm:

Space complexity Time complexity Asymptotic notation- Big-O, Theta and Omega, Finding complexity using step count method

What is Complexity Analysis?

Complexity analysis is the process of determining how the execution time or memory usage of an algorithm grows as the input size increases.

Time Complexity: Measures the amount of time an algorithm takes.

Space Complexity: Measures the amount of memory an algorithm uses.

The input size is generally represented by **n**.

- Complexity analysis helps us understand how much time or space an algorithm will need as the size of the input grows.
- The input is our data, and the algorithms are the steps we take to process that data.
- We estimate the time required to solve a problem or the amount of memory necessary based on our approach.

Why Complexity Analysis?

Instead of measuring execution time on a particular computer, we analyze the growth of an algorithm using mathematical functions.

Example: Although both algorithms solve the same problem, Algorithm B grows much more slowly.

Input Size (n)	Algorithm A	Algorithm B
10	100 operations	30 operations
100	10,000 operations	600 operations
1000	1,000,000 operations	9,000 operations

Role of f(n) and G(n)

In asymptotic analysis, two functions are commonly used:

- **f(n)** → Actual running time (or actual growth function) of an algorithm.
- **g(n)** → Reference or comparison function used to describe the growth rate.

For example,

Suppose

$$f(n) = 3n^2 + 5n + 10$$

We compare it with

$$g(n) = n^2$$

Since the highest-order term dominates,

$$3n^2 + 5n + 10 \approx n^2$$

Thus,

$$f(n) = O(n^2)$$

Why Ignore Constants?

Suppose

Algorithm A:

$$f(n) = 100n$$

Algorithm B:

$$f(n) = n^2$$

For

n = 10

- A =1000
- B =100

Algorithm B seems better.

But for

n =1000

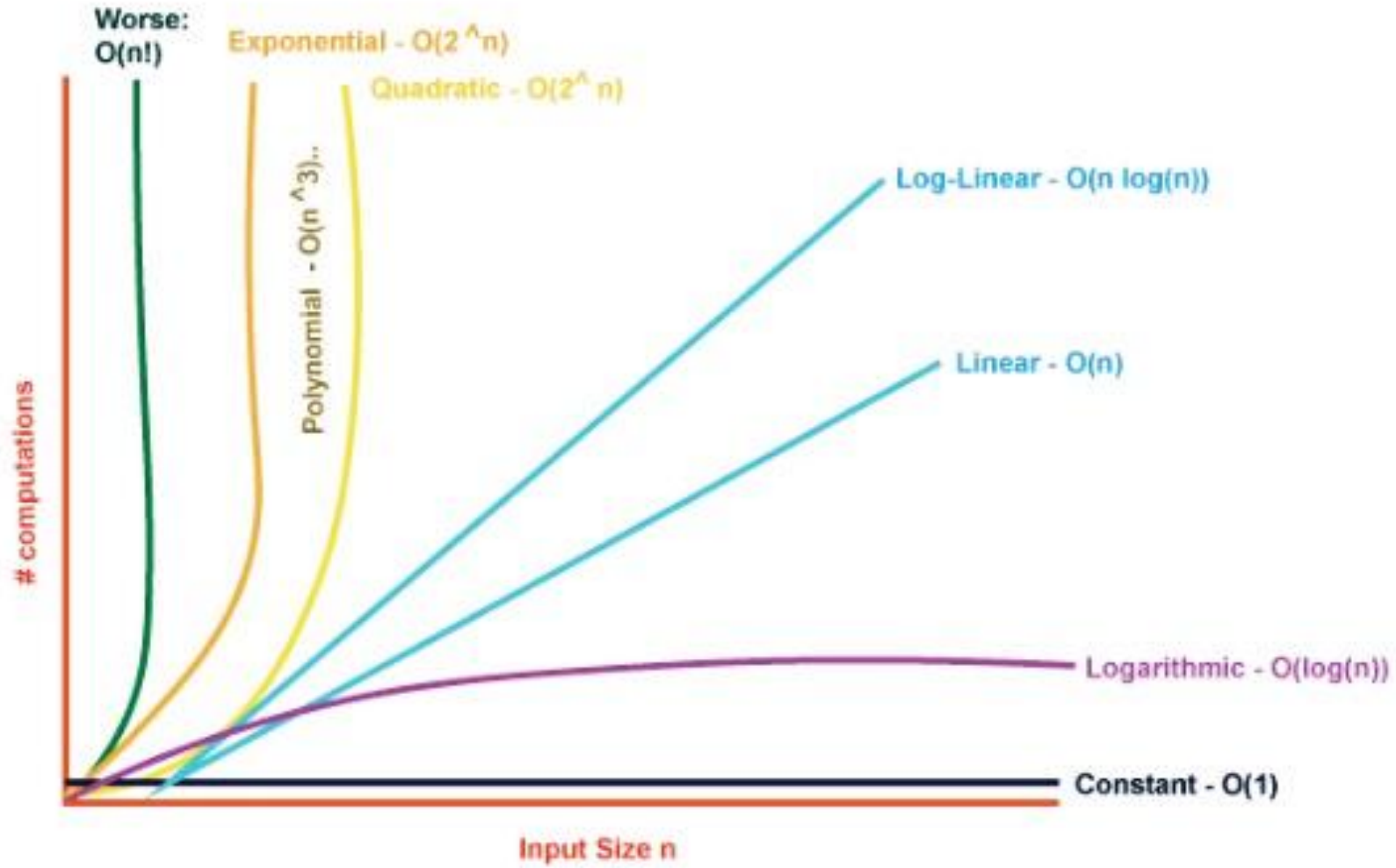
- A =100000
- B =1000000

Algorithm A becomes much faster.

Hence asymptotic analysis focuses only on growth for **large n**.

Asymptotic Notations

- There are several types of asymptotic notations, including Big O, Big Omega, and Big Theta. Each notation provides a different perspective on the growth rate of a function.
- **Big-Oh (O) Notation:** It represents the maximum amount of time or space required by an algorithm considering all input values. It represents the upper limit of an algorithm's execution time or space, offering insight into its worst-case complexity.
- **Big-Omega (Ω) notation:** It represents the minimum amount of time or space required by an algorithm considering all input values.
- **Big-Theta (Θ) notation:** It represents the upper and the lower bound of the running time of an algorithm, it is used for analyzing the average-case complexity of an algorithm.


Worse:
O(n!)
Exponential - O(2 ^n)
Quadratic - O(2^ n)
Polynomial - O(n ^3)..
Log-Linear - O(n log(n))
Linear - O(n)
Logarithmic - O(log(n))
Constant - O(1)
computations
Input Size n

Time Complexity?

- ***Time complexity measures the amount of time an algorithm takes to run as a function of the input size.***
- Algorithms with lower time complexity are generally more desirable as they can handle larger input sizes with reasonable runtimes.
- Let's understand the common time complexities with code examples

1. Constant Time: O(1)

The execution time does not depend on the size of the input. No matter how large the input is, the algorithm takes the same amount of time to complete.

```
public class ConstantTimeExample {
    public static void printFirstElement(int[] array) {
        if (array.length > 0) {
            System.out.println(array[0]);
        }
    }
}
```

Time Complexity

- *The field of computer science, which studies efficiency of algorithms, is known as analysis of algorithms.*
- Time Complexity is the amount of time taken by the algorithm to run. It measures the time taken to execute each statement of code in an algorithm.
- **Time Complexity** can be calculated by using Two types of methods. They are:
- Step Count Method
- Asymptotic Notation

Big O Notation	Name	Example(s)
$O(1)$	Constant	# Odd or Even number, # Look-up table (on average)
$O(\log n)$	Logarithmic	# Finding element on sorted array with **binary search**
$O(n)$	Linear	# Find max element in unsorted array, # Duplicate elements in array with Hash Map
$O(n \log n)$	Linearithmic	# Sorting elements in array with **merge sort**
$O(n^2)$	Quadratic	# Duplicate elements in array **(naïve)**, # Sorting array with **bubble sort**
$O(n^3)$	Cubic	# 3 variables equation solver
$O(2^n)$	Exponential	# Find all subsets
$O(n!)$	Factorial	# Find all permutations of a given set/string

Space Complexity

- Problem-solving using computer requires memory to hold temporary data or final result while the program is in execution. The amount of memory required by the algorithm to solve given problem is called **space complexity** of the algorithm.
- The space complexity of an algorithm quantifies the amount of space taken by an algorithm to run as a function of the length of the input
- To estimate the memory requirement we need to focus on two parts:
- **(1) A fixed part:** It is independent of the input size. It includes memory for instructions (code), constants, variables, etc.
- **(2) A variable part:** It is dependent on the input size. It includes memory for recursion stack, referenced variables, etc.

Asymptotic Notation

- Asymptotic Notations are programming language representations that allow you to analyze an algorithm's running time by identifying its behavior as its input size grows. This is also referred to as an algorithm's growth rate.
- **Asymptotic notation of an algorithm is a mathematical representation of its complexity.**
- Asymptotic notations are classified into three types:
- Big-Oh (O) notation
- Big Omega (Ω) notation
- Big Theta (Θ) notation

Asymptotic Notation

- O notation: asymptotic "less than": **upper bound**

$f(n)=O(g(n))$ implies: $f(n)$ "$\leq$" $g(n)$ **worst-case complexity**

- Ω notation: asymptotic "greater than": **Lower bound**

$f(n)=\Omega(g(n))$ implies: $f(n)$ "$\geq$" $g(n)$ **Best case Complexity**

- Θ notation: asymptotic "equality":

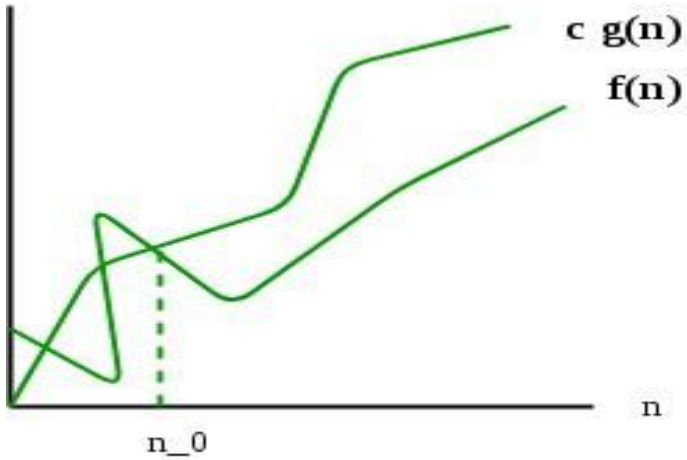
$f(n)=\Theta(g(n))$ implies: $f(n)$ "$=$" $g(n)$ **average-case Complexity**

Big Oh Notation

- Big-O notation represents the **upper bound of the running time** of an algorithm. Therefore, it gives the **worst-case complexity of an algorithm.**
- It is the most widely used notation for Asymptotic analysis. It specifies the upper bound of a function. The maximum time required by an algorithm or the worst-case time complexity.
- It returns the highest possible output value(big-O) for a given input.
- Big-Oh(Worst Case) It is defined as the condition that allows an algorithm to complete statement execution in the longest amount of time possible.

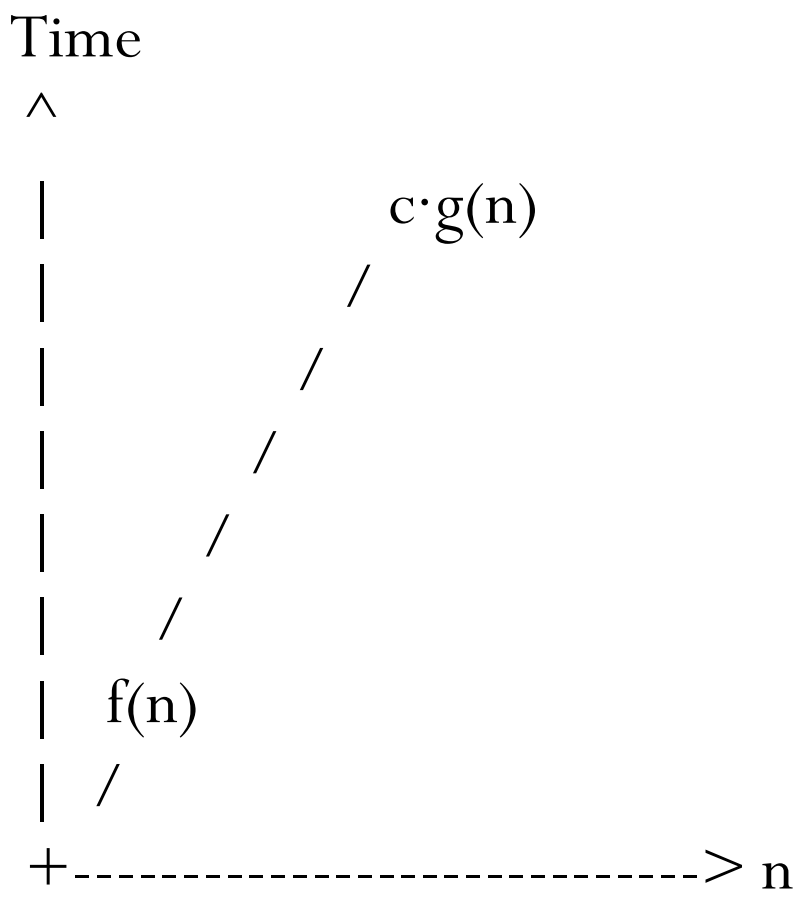
Big Oh Notation

- If f(n) describes the running time of an algorithm, f(n) is O(g(n)) if there exist a positive constant C and n0 such that, $0 \leq f(n) \leq cg(n)$ for all $n \geq n0$
- **It returns the highest possible output value (big-O)for a given input.**
- **The execution time serves as an upper bound on the algorithm's time complexity.**



f(n) = O(g(n))

Big O Notations



f(n) always remains below c·g(n).

Example

- Consider the following f(n) and g(n)…

f(n) = 3n + 2

g(n) = n

If we want to represent **f(n)** as **O(g(n))** then it must satisfy **f(n) <= C g(n)** for all values of **C > 0** and $\mathbf{n_0}$**>= 1**

f(n) <= C g(n)

⇒3n + 2 <= C n

Above condition is always TRUE for all values of **C = 4** and **n >= 2**.

By using Big - Oh notation we can represent the time complexity as follows…

3n + 2 = O(n)

Example 1

$$f(n) = 4n + 7$$

Take

$$g(n) = n$$

Need to prove

$$4n + 7 \leq cn$$

Choose

$$c = 5$$

For

$$n \geq 7$$

$$4n + 7 \leq 5n$$

Hence

$$4n + 7 = O(n)$$

Example 2

$$f(n) = 5n^2 + 3n + 2$$

Dominant term

$$n^2$$

Therefore

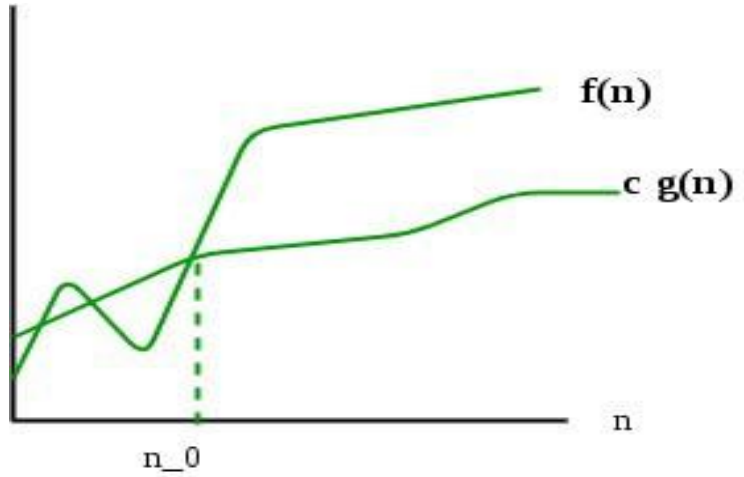
$$f(n) = O(n^2)$$

Omega Notation (Ω-Notation):

- Omega notation represents the lower bound of the running time of an algorithm. Thus, it provides the best case complexity of an algorithm.
- **The execution time serves as a lower bound on the algorithm's time complexity.**
- **It is defined as the condition that allows an algorithm to complete statement execution in the shortest amount of time.**

Mathematical Representation of Omega Notation (Ω-Notation):

- Let g and f be the function from the set of natural numbers to itself. The function f is said to be $\Omega(g)$, if there is a constant c > 0 and a natural number n0 such that $c*g(n) \leq f(n)$ for all $n \geq n0$
- $\Omega(g(n)) = \{ f(n)$: there exist positive constants c and n0 such that $0 \leq cg(n) \leq f(n)$ for all $n \geq n0 \}$



f(n) = Omega(g(n))

2. Big-Omega (Lower Bound)

Definition

Big-Omega gives the **minimum growth rate**.

It tells us

> "The algorithm will take at least this much time."

Mathematically,

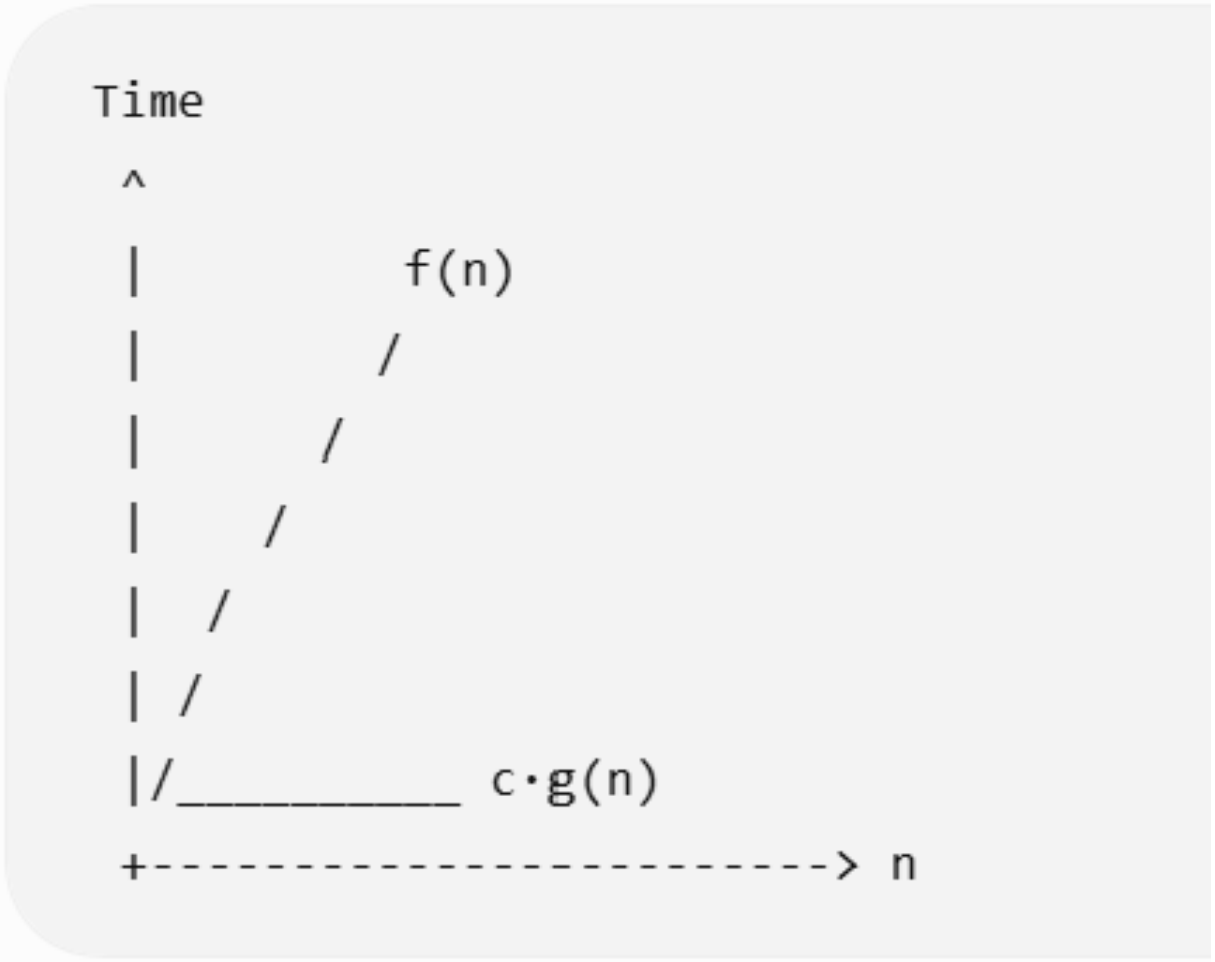
$$f(n) = \Omega(g(n))$$

if

$$f(n) \geq cg(n)$$

for sufficiently large n.

Graph



f(n) always stays above c·g(n).

Example

$$f(n) = 6n + 8$$

Compare with

$$g(n) = n$$

Choose

$$c = 6$$

Then

$$6n + 8 \geq 6n$$

Therefore

$$f(n) = \Omega(n)$$

Example

- Consider the following f(n) and g(n)…

  f(n) = 3n + 2

  g(n) = n

  If we want to represent **f(n)** as **Ω(g(n))** then it must satisfy **f(n) >= C g(n)** for all values of **C > 0** and $\mathbf{n_0}$**>= 1**

  f(n) >= C g(n)

  ⇒3n + 2 >= C n

  Above condition is always TRUE for all values of

C = 1 and **n >= 1**.

By using Big - Omega notation we can represent the time complexity as follows…

3n + 2 = Ω(n)

Theta Notation (Θ-Notation):

- Theta notation encloses the function from above and below. Since it represents the upper and the lower bound of the running time of an algorithm, it is used for analyzing the **average-case complexity of an algorithm**.
- Theta (Average Case) can add the running times for each possible input combination and take the average in the average case.

3. Big-Theta (Tight Bound)

Definition

Big-Theta means

Upper Bound + Lower Bound together.

Mathematically,

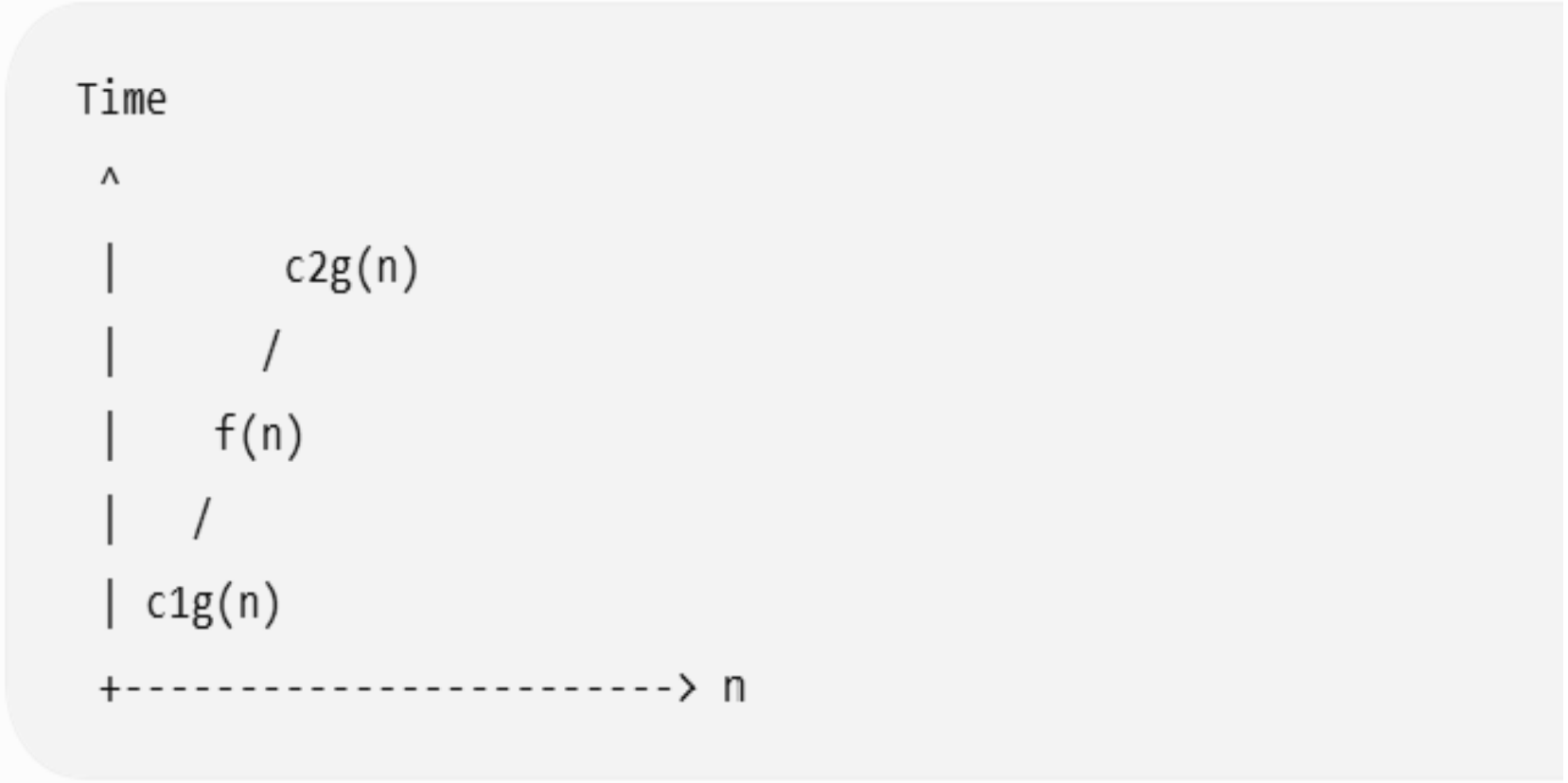
$$f(n) = \Theta(g(n))$$

if

$$c_1 g(n) \leq f(n) \leq c_2 g(n)$$

for sufficiently large n.

Graph



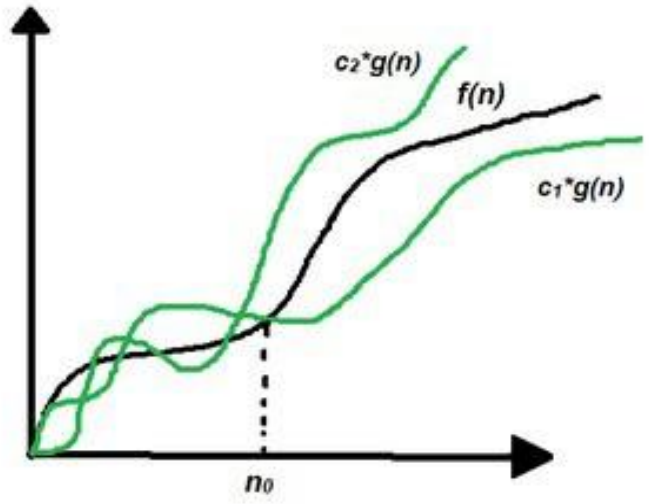
The function remains between two multiples of g(n).

Mathematical Representation of Theta notation:

- Let g and f be the function from the set of natural numbers to itself. The function f is said to be Θ(g), if there are constants c1, c2 > 0 and a natural number n0 such that

  c1* g(n) ≤ f(n) ≤ c2 * g(n) for all n ≥ n0

- Θ (g(n)) = {f(n) there exist positive constants c1 c2 and n0 such that 0 ≤ c1 * g(n) ≤ f(n) ≤ c2 * g(n) for all n ≥ n0}



Example

$$f(n) = 7n + 10$$

Compare with

$$g(n) = n$$

Upper bound

$$7n + 10 \leq 8n$$

Lower bound

$$7n + 10 \geq 7n$$

Hence

$$f(n) = \Theta(n)$$

Example

- Consider the following f(n) and g(n)…

f(n) = 3n + 2

g(n) = n

If we want to represent **f(n)** as **Θ(g(n))** then it must satisfy **C_1 g(n) <= f(n) <= C_2 g(n)** for all values of **C_1 > 0, C_2 > 0** and **n_0>= 1**

C_1 g(n) <= f(n) <= C_2 g(n)

$\Rightarrow C_1$ n <= 3n + 2 <= C_2 n

Above condition is always TRUE for all values of **C_1 = 1, C_2 = 4** and **n >= 2**.

By using Big - Theta notation we can represent the time compexity as follows…

3n + 2 = Θ(n)

Common Growth Rates

Complexity	Name	Example
O(1)	Constant	Array access
O(log n)	Logarithmic	Binary Search
O(n)	Linear	Linear Search
O(n log n)	Linearithmic	Merge Sort, Heap Sort
$O(n^2)$	Quadratic	Bubble Sort, Selection Sort
$O(n^3)$	Cubic	Matrix multiplication (basic)
$O(2^n)$	Exponential	Recursive Fibonacci
O(n!)	Factorial	Traveling Salesman (brute force)

How to measure complexity?

- The complexity of an algorithm can be measured in three ways:
- 1. Time Complexity
- The **time complexity** of an algorithm is defined as the amount of time taken by an algorithm to run as a function of the length of the input. Note that the time to run is a **function of the length of the input** and not the actual execution time of the machine on which the algorithm is running on.
- To estimate the time complexity, we need to consider the cost of each fundamental instruction and the number of times the instruction is executed.
- If we have statements with basic operations like **comparisons, return statements, assignments, and reading a variable.** We can assume they take **constant time each O(1).**

Statement 1: int a=5; // reading a variable

statement 2; if(a==5) return true; // return statement

statement 3; int x= 4>5 ? 1:0; // comparison

statement 4; bool flag=true; // Assignment

This is the result of calculating the overall time complexity.

total time = time(statement1) + time(statement2) + … time (statementN)

Assuming that **n is the size** of the input, **let's use T(n)** to represent the **overall time** and t to represent the **amount of time** that a **statement or collection** of statements takes to **execute.**

T(n) = t(statement1) + t(statement2) + ... + t(statementN);

Overall, T(n)= O(1), which means constant complexity.

For any loop, we find out the runtime of the block inside them and multiply it by the number of times the program will repeat the loop.

```
for (int i = 0; i < n; i++) {
   cout << "Hello" << endl;
}
```

For the above example, the loop will execute **n times,** and it will print **"Hello" N number of times.** so the time taken to run this program is:

T(N)= n *(t(cout statement))

= n * O(1)

=O(n), **Linear complexity.**

For 2D arrays, we would have nested loop concepts, which means a loop inside a loop.

```
for (int i = 0; i < n; i++) {
  for (int j = 0; j < m; j++) {
      cout << “Hello" << endl;
  }
}
```

For the above example, the **cout** statement will execute **n*m times,** and it will print **“Hello" N*M number of times.** so the time taken to run this program is:

T(N)= n * m *(t(cout statement))

= n * m * O(1)

=O(n*m), **Quadratic Complexity.**

Time and Space Complexity for addition of two numbers

- The time complexity of the problem addition of two numbers in C/C++ is O(1) because it doesn't depend on the input values.
- The space is only taken to store the two variables and their sum hence the space complexity for a program to find the addition of two numbers in C is O(1).

1. Big O Notation (O) — Upper Bound

Meaning: Describes the **maximum** time or space an algorithm can take.

Symbolic analogy: <= (less than or equal to).

Use: When we say an algorithm is **O(f(n))**, we mean *it won't grow faster than* f(n) as input size increases.

Example: If an algorithm takes at most $5n^2 + 3n + 10$ steps, it is **$O(n^2)$**.

2. Big Omega Notation (Ω) — Lower Bound

Meaning: Describes the **minimum** time or space required by an algorithm.

Symbolic analogy: >= (greater than or equal to).

Use: When we say an algorithm is **Ω(f(n))**, we mean *it will take at least* f(n) time for large inputs.

Example: If an algorithm always takes at least n steps, it is **Ω(n)**.

3. Big Theta Notation (Θ) — Tight Bound

Meaning: Describes the **exact** growth rate (both upper and lower bound).

Symbolic analogy: == (equal to).

Use: When an algorithm is **Θ(f(n))**, it means its time complexity grows *exactly like* f(n) for large inputs.

Example: If an algorithm always takes between 3n and 5n steps, it is **Θ(n)**.

2. Space Complexity :

The amount of memory required by the algorithm to solve a given problem is called the space complexity of the algorithm. Problem-solving using a computer requires memory to hold temporary data or final result while the program is in execution.

How is space complexity computed?

The space Complexity of an algorithm is the total space taken by the algorithm with respect to the input size. Space complexity includes both Auxiliary space and space used by input.

Space complexity is a parallel concept to time complexity. If we need to create an array of size n, this will **require O(n) space**. If we create a two-dimensional array of size **n*n**, this will require **$O(n^2)$ space.**

In recursive calls stack space also counts.

```
int add (int n){
    if (n <= 0){
        return 0;
    }
    return n + add (n-1);
}
```

Here each call add a level to the stack :

1. add(4)
2. -> add(3)
3. -> add(2)
4. -> add(1)
5. -> add(0)

Each of these calls is added to call stack and takes up actual memory.

So it takes O(n) space.

However, just because you have n calls total doesn't mean **it takes O(n) space.**

Look at the below function :

```
int addSequence (int n){
    int sum = 0;
    for (int i = 0; i < n; i++){
        sum += pairSum(i, i+1);
    }
    return sum;
}
int pairSum(int x, int y){
    return x + y;
}
```

There will be roughly O(n) calls to pairSum. However, those calls do not exist simultaneously on the call stack, so you only need O(1) space.

S.No.	Big Oh (O)	Big Omega (Ω)	Big Theta (Θ)
1.	It is like (<=) rate of growth of an algorithm is less than or equal to a specific value.	It is like (>=) rate of growth is greater than or equal to a specified value.	It is like (==) meaning the rate of growth is equal to a specified value.
2.	The upper bound of algorithm is represented by Big O notation. Only the above function is bounded by Big O. Asymptotic upper bound is given by Big O notation.	The algorithm's lower bound is represented by Omega notation. The asymptotic lower bound is given by Omega notation.	The bounding of function from above and below is represented by theta notation. The exact asymptotic behavior is done by this theta notation.
3.	Big oh (O) – Upper Bound	Big Omega (Ω) – Lower Bound	Big Theta (Θ) – Tight Bound
4.	It is define as upper bound and upper bound on an algorithm is the most amount of time required (the worst case performance).	It is define as lower bound and lower bound on an algorithm is the least amount of time required (the most efficient way possible, in other words best case).	It is define as tightest bound and tightest bound is the best of all the worst case times that the algorithm can take.
5.	Mathematically: Big Oh is 0 <= f(n) <= Cg(n) for all n >= n0	Mathematically: Big Omega is 0 <= Cg(n) <= f(n) for all n >= n0	Mathematically – Big Theta is 0 <= C2g(n) <= f(n) <= C1g(n) for n >= n0

Step Count Method

- The step count method is one of the method to analyze the algorithm. In this method, we count number of times one instruction is executing. From that we will try to find the complexity of the algorithm.
- It works by counting exactly how many times each individual instruction or line of code executes relative to the input size n. By summing up these executions, you construct a time function T(n) and simplify it using **Big O notation** to find the algorithm's runtime growth.
- Suppose we have one algorithm to perform sequential search. Suppose each instruction will take c1, c2, …. amount of time to execute, then we will try to find out the time complexity of this algorithm
- The step Count method is also called as **Frequency Count method**.

Time Complexity

Algorithm	Number of times	Cost
seqSearch(arr, n, key) i := 0 while i < n, do if arr[i] = key, then break end if done return i	1 n+1 n 0/1 1	c1 c2 c3 c4 c5

Now if we add the cost by multiplying the number of times it is executed, (considering the worst case situation), we will get

$$Cost=c1+(n+1)c\ 2+nc3+c\ 4+c\ 5$$

$$Cost=c1+nc\ 2+c2+nc\ 3+c\ 4+c5$$

$$Cost=n(c\ 2+c3)+c\ 1+c\ 4+c5$$

$$Cost=n(c\ 2+c3)+C$$

Considering the c1 + c4 + c5 is C, so the final equation is like straight line y = mx + b. So we can say that the function is linear. The complexity will be O(n).

To use this method, you look at every line of code and assign an execution count (frequency) based on standard execution rules:

Declarations and Comments: Executed **0 times** (they do not take processing time).

- **Simple Assignments & Returns**: Executed **1 time**.
- **Sequential Statements**: Executed **1 time** each.
- **Single Loops (for / while)**: The condition line executes **(n + 1 times** (the extra 1 is for the final check that fails and breaks the loop). The body statements inside that loop execute exactly **n times**.
- **Nested Loops**: Multiply the frequencies of the outer loop iterations by the inner loop iterations

1. Linear Time Example: Summing an Array

```
int sum(int arr[], int n)
{
        int s = 0;                          // Line 1
       for (int i = 0; i < n; i++)          // Line 2
       {
            s = s + arr[i];                 // Line 3
       }
      return s;                             // Line 4
}
```

Frequency Table Analysis:

Line #	Code Statement	Frequency (Cost)	Explanation
Line 1	`int s = 0;`	1	Executes exactly once at startup.
Line 2	`for (int i = 0; i < n; i++)`	$n + 1$	Runs n times for valid loop runs + 1 time for the false condition check.
Line 3	`s = s + arr[i];`	n	Executes exactly n times inside the loop.
Line 4	`return s;`	1	Runs exactly once when the function terminates.

Deriving Time Complexity:

1. Sum all frequencies to build your total time equation:

$$T(n) = 1 + (n + 1) + n + 1$$

2. Simplify the polynomial:

$$T(n) = 2n + 3$$

3. Drop constants and lower-order terms to find the Big O notation:

$$\mathbf{O(n)} \quad \text{(Linear Time)}$$

Algorithmic Strategies-

Introduction to algorithm design strategies- Divide and Conquer, and Greedy strategy.

Algorithm design Strategies

- Divide and conquer algorithms – divide the problem into smaller subproblems of the same type; solve those smaller problems, and combine those solutions to solve the original problem. Eg. Binary Search, Quick Sort, Merge Sort

- Greedy algorithms – find an optimal solution at the local level with the intent of finding an optimal solution for the whole problem. Eg. Job Scheduling, Activity selection, Fractional Knapsack, MST, Huffman Code

Algorithm design Strategies

- Backtracking algorithms – divide the problem into subproblems, each which can be attempted to be solved; however, if the desired solution is not reached, move backwards in the problem until a path is found that moves it forward. Eg. 8 Queen Problem, Sum of Subset, Graph Coloring
- Dynamic programming algorithms – break a complex problem into a collection of simpler subproblems, then solve each of those subproblems only once, storing their solution for future use instead of re-computing their solutions. Eg. OBST,0/1 knapsack problems

Algorithm design Strategies

- Brute force algorithms – try all possible solutions until a satisfactory solution is found.eg. Sequential Search, Bubble Sort
- Randomized algorithms – use a random number at least once during the computation to find a solution to the problem. Eg. Randomized quick sort
- Recursive algorithms – solve the lowest and simplest version of a problem to then solve increasingly larger versions of the problem until the solution to the original problem is found. Eg. Factorial of number, merge sort, quick sort

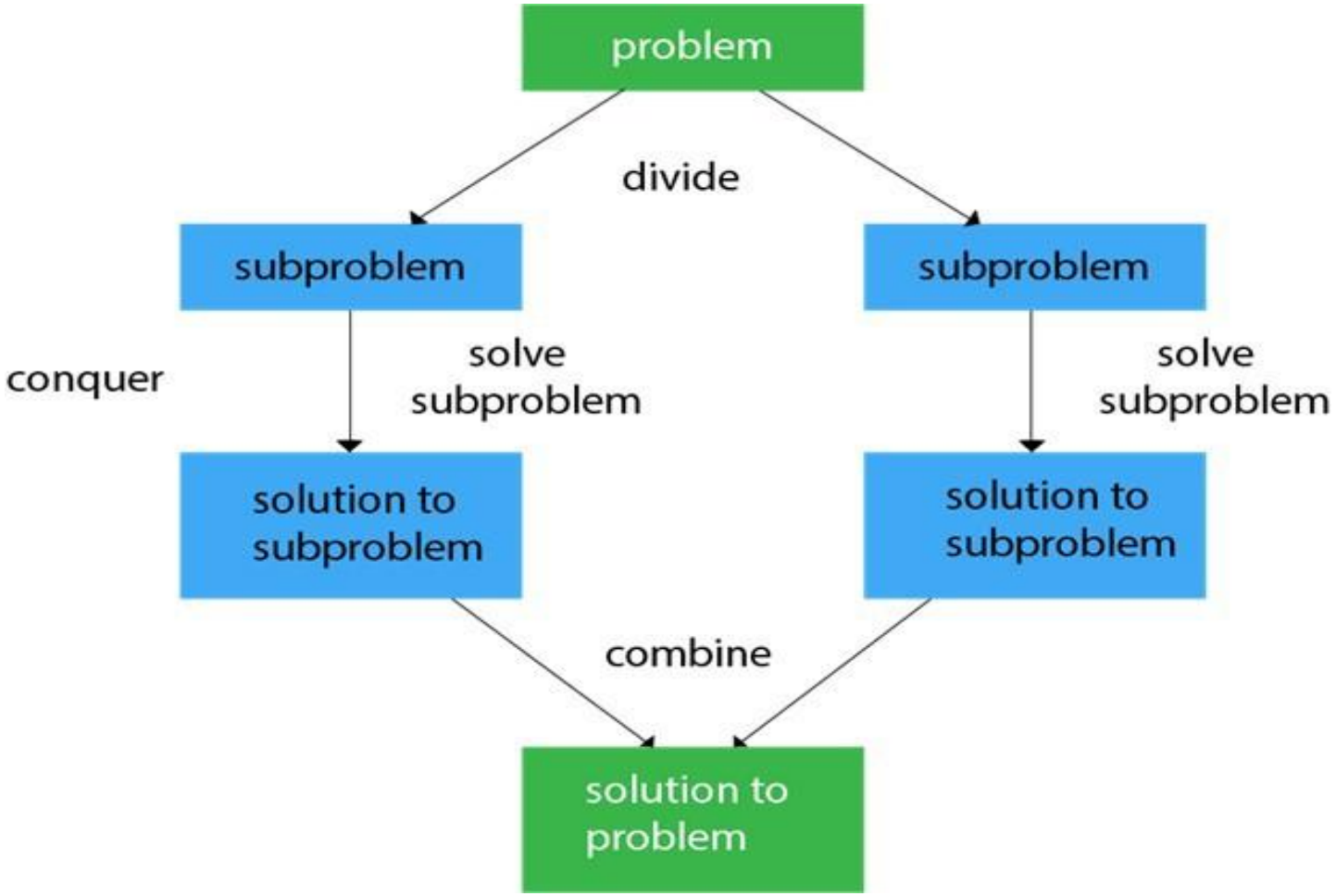
Divide and Conquer

Divide and Conquer is an algorithmic paradigm in which the problem is solved using the Divide, Conquer, and Combine strategy.

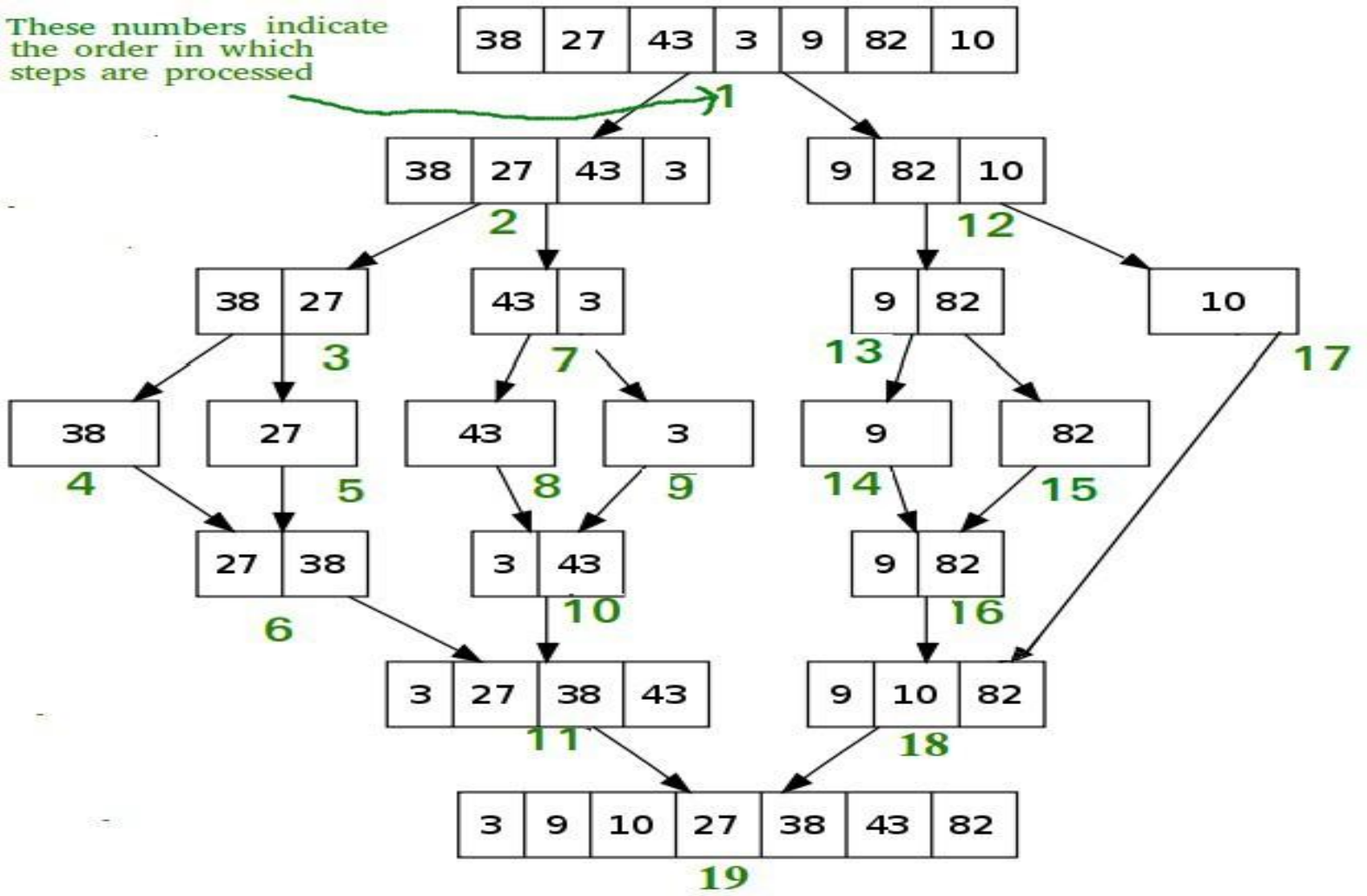
- **Divide** the original problem into a set of subproblems.
- **Conquer:** Solve every subproblem individually, recursively.
- **Combine:** Put together the solutions of the subproblems to get the solution to the whole problem.

Examples:

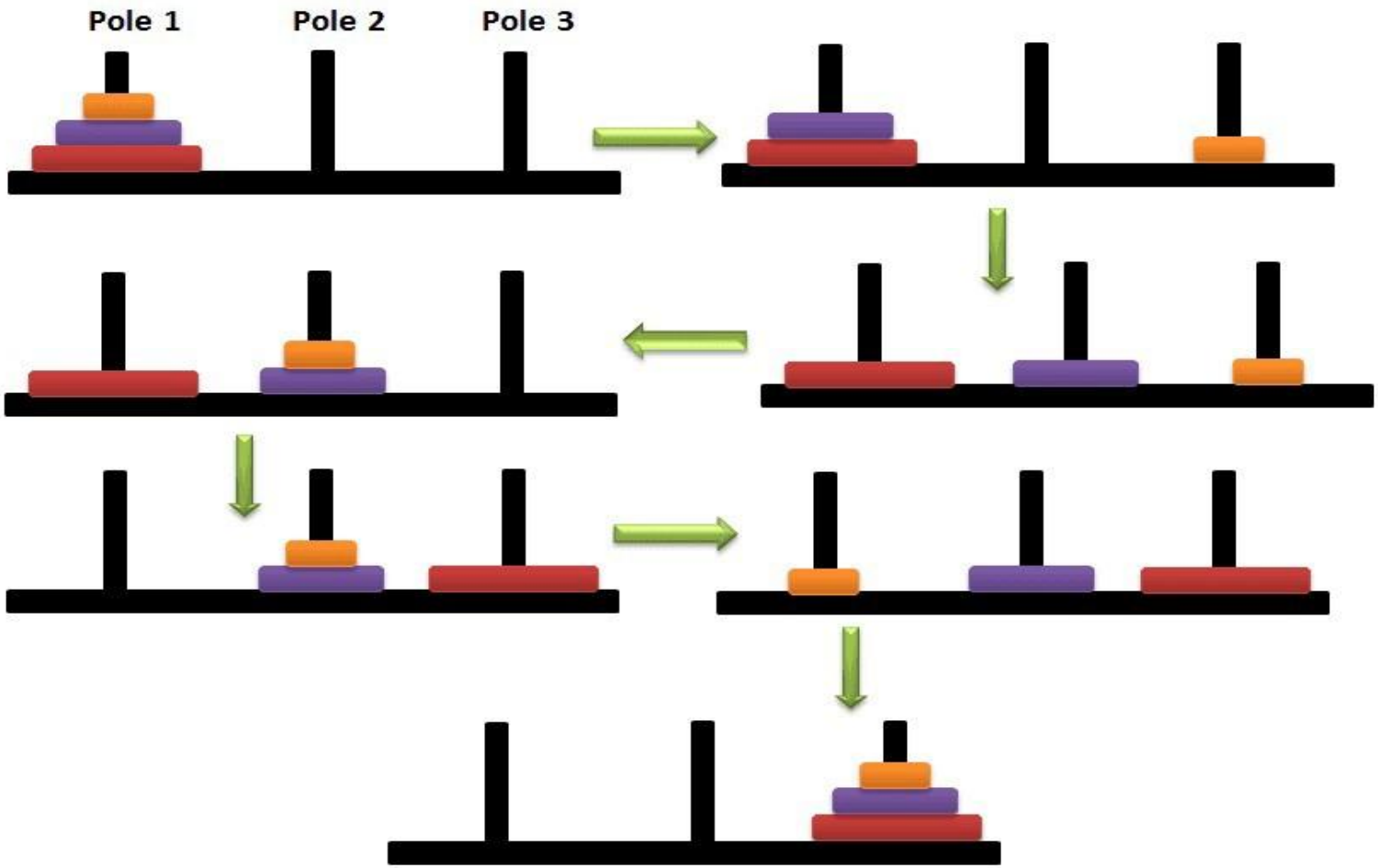
- Binary Search
- Sorting (merge sort, quick sort)
- Tower of Hanoi.


problem
divide
subproblem
subproblem
conquer
solve
subproblem
solve
subproblem
solution to
subproblem
solution to
subproblem
combine
solution to
problem

Merge Sort


These numbers indicate the order in which steps are processed
38 27 43 3 9 82 10
1
38 27 43 3
2
9 82 10
12
38 27
3
43 3
7
9 82
13
10
17
38
4
27
5
43
8
3
9
9
14
82
15
27 38
6
3 43
10
9 82
16
3 27 38 43
11
9 10 82
18
3 9 10 27 38 43 82
19

Tower of Hanoi



Advantages of Divide and Conquer

- Divide and Conquer tend to successfully solve one of the biggest problems, such as the Tower of Hanoi, a mathematical puzzle. It is challenging to solve complicated problems for which you have no basic idea, but with the help of the divide and conquer approach, it has lessened the effort as it works on dividing the main problem into two halves and then solve them recursively. This algorithm is much faster than other algorithms.
- It efficiently uses cache memory without occupying much space because it solves simple subproblems within the cache memory instead of accessing the slower main memory.
- Since these algorithms inhibit parallelism, it does not involve any modification and is handled by systems incorporating parallel processing

Disadvantages of Divide and Conquer

- Since most of its algorithms are designed by incorporating recursion, so it necessitates high memory management.
- An explicit stack may overuse the space.
- It may even crash the system if the recursion is performed rigorously greater than the stack present in the system .

Greedy Algorithm

- Greedy Algorithm is defined as a method for solving **optimization problems** by taking decisions that result in the most evident and immediate benefit irrespective of the final outcome. It works for cases where **minimization** or **maximization** leads to the required solution.
- Think of a greedy algorithm like planning a road trip. At every intersection, you pick the shortest-looking path to reach your destination quickly. It seems smart at the moment, right? But sometimes that path leads to traffic or a dead end. That's how a greedy algorithm works—it makes the best immediate choice, without thinking ahead. It's fast and simple but doesn't always guarantee the best overall solution.

Characteristics of Greedy algorithm

- For a problem to be solved using the Greedy approach, it must follow a few major characteristics:
- There is an ordered list of resources(profit, cost, value, etc.)
- Maximum of all the resources(max profit, max value, etc.) are taken
- For example, in the fractional knapsack problem, the maximum value/weight is taken first according to available capacity.

Greedy Algorithm Example:

Some Famous problems that exhibit **Optimal substructure property** and can be solved using Greedy approach are –

1) Job sequencing Problem:

Greedily choose the jobs with maximum profit first, by sorting the jobs in decreasing order of their profit. This would help to maximize the total profit as choosing the job with maximum profit for every time slot will eventually maximize the total profit

2) Prim's algorithm to find Minimum Spanning Tree:

It starts with an empty spanning tree. The idea is to maintain two sets of vertices. The first set contains the vertices already included in the MST, the other set contains the vertices not yet included. At every step, it considers all the edges that connect the two sets and picks the minimum weight edge from these edges. After picking the edge, it moves the other endpoint of the edge to the set containing MST.

Greedy Strategy - Examples

Examples:
- Activity Selection
- Fractional Knapsack
- Prim's Algorithm (MST)
- Huffman Coding
- Dijkstra's Algorithm

Real-Life Analogy:
- Picking the cheapest item available each time while shopping.

Example of Problem-Solving Using a Greedy Algorithm

Problem: You have to make a change of an amount using the smallest possible number of coins.

Amount: $22

Available coins are

- $5 coin
- $2 coin
- $1 coin

There is no limit to the number of each coin you can use.

Solution:

Create an empty solution-set = { }. Available coins are {5, 2, 1}.

We are supposed to find the sum = 22. Let's start with sum = 0.

Always select the coin with the largest value (i.e. 5) until the sum > 22.

(When we select the largest value at each step, we hope to reach the destination faster. This concept is called greedy choice property.)

In the first iteration, solution-set = {5} and sum = 5.

In the second iteration, solution-set = {5, 5} and sum = 10.

In the third iteration, solution-set = {5, 5, 5} and sum = 15.

In the fourth iteration, solution-set = {5, 5, 5, 5} and sum = 20. (We cannot select 5 here because if we do so, sum = 20 and we want to make 22. So, we select the 2nd largest item which is 2).

In the fifth iteration, select 2. Now sum = 22 and solution-set = {5, 5, 5, 5, 2}.

Example- Fractional Knapsack

Problem: Maximize value by putting items in a knapsack where fractional parts of an item are allowed.

Approach:

Calculate the value/weight for each item.

Sort items by this ratio in descending order.

Pick as much as possible from the highest value/weight item

Knapsack Problem

(Q8) $w = 5, 10, 15, 20$

Total Weight of Knapsack = 25

$v = 50, 60, 120, 100.$

Item	w_i	v_i	$P_i = v_i/w_i$
1	5	50	50/5=10
2	10	60	60/10 = 6
3	15	120	120/15 = 8
4	20	100	100/20 = 5

Arranging values of P_i obtained in decreasing order, in order to get optimal solution for Knapsack Problem.

Item	w_i	v_i	P_i
1	5	50	10
3	15	120	8
2	10	60	6
4	20	100	5

Total capacity is 25, so adding Items 1, 3 gives $5+15=20$.

$25-20=5$ so, item 2 will be added as $\frac{60}{10} \times 5 = 30$.

Maximum Profit obtained $\Rightarrow 50+120+30$

$\Rightarrow 200.$

Prims Algorithm

